the hardware sector size, most often 512 bytes) when it encounters errors on a drive, and then skips the error-prone sectors, so that it gets as much of the drive as is possible. '*dd_rescue* ' will not abort on errors unless one specifies a number of errors (using the *–e* option) after which to abort. Also, '*dd_rescue* ' can start from the end of the disk/partition and move backwards. Finally, with '*dd_rescue* ', you can optionally write a log file, which can provide useful information about the location of bad sectors. The basic usage of '*dd_rescue* ' is shown below:

```
dd_rescue <input file> <out file>
```

For example:

```
dd_rescue    /dev/hdc      image.ddrescue
```

dcfldd

'*dcfldd* ' is an enhanced version of '*dd* ' with the following useful additional features for forensics:

- On-the-fly hashing of the transmitted data
- Progress updation for the user
- Splitting of output in multiple files
- Simultaneous output to multiple files at the same time
- Wiping the disk with a known pattern
- Image verification for verifying an image against the original source/pattern

Basic usage for '*dcfldd* ' is '*dcfldd <options> if=<input media> of=<image_file>*', where *<input media>* specifies the input media and *<image_file>* specifies the image file, or the prefix for multiple image files. Some of the useful options are:

split=<bytes>	Specifies the size of each of the output image files.
vf=<file>	Specifies the image file that needs to be verified against the input.
splitformat=<text>	Specifies the format for multiple image files when using splitting.
hash=<names>	Specifies one or more (a comma-separated list) of hash algorithms, such as md5, sha1, etc.
hashwindow=<bytes>	Specifies the number of bytes of input media for calculation of hash.
<hash algorithm>log=<file>	Specifies output files for hash calculations of a particular hash algorithm.
conv=<keywords>	Specifies conversions of input media according to a comma-separated list of keywords, such as 'noerror ' (continue after read errors), 'ucase ' (change lower to upper case), etc.

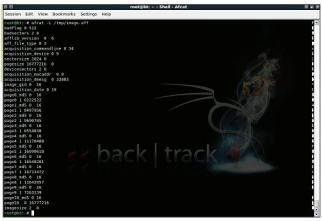

Figure 7: Output of 'afcat' command line utility

For example:

```
dcfldd if=/dev/hdc hash=md5 hashwindow=10G md5log=md5.txt
hashconv=after bs=512 conv=noerror,sync split=10G splitformat=aa
of=image.dd
```

In this example, 10 gigabytes will be read from the input drive and written to the file *image.dd.aa*. Then it will read the next 10 gigabytes and write it to file *image.dd.ab*. Also, an MD5 hash of every 10-gigabyte chunk on the input drive will be stored in the file called md5.txt. The block size is set to 512 bytes, and in the event of read errors, *dcfldd* will write zeros to the output and continue reading.

Aimage

'*Aimage*' is an advanced forensics format (AFF) imaging tool with intelligent error recovery, compression and verification. AFF is an extensible open format for the storage of disk images and related forensic meta-data. The ability to store relevant metadata, such as the serial number of the drive, date of imaging, etc, inside the image itself, is a huge advantage, and is not possible with raw imaging tools such as '*dd* '. Many popular forensics tools such as Sleuthkit and Autopsy have support for analysing images stored in AFF format.

AFF is a segmented archive file specification (Figure 3) where each file consists of an AFF file header followed by one or more file segments, and ends with a special directory segment that lists all the segments of the file, and their offset from the file's start. These segments are used to store all information, including the image itself, and image metadata, inside the AFF file. Further, each AFF segment consists of a header, variable-length segment name, a 32-bit flag, variable-length data payload and segment footer (Figure 4). The length of the entire segment is stored in the header and footer, for rapid bi-directional seeking through an AFF image file. Segments can be between 32 bytes and 2^32-1 bytes long. There can be metadata segments, which